

Flask Online Shop

A Deep Dive into a Flask + Stripe Storefront

Portfolio explainer, written for a reader with no prior background

The one-paragraph version

Flask Online Shop is a small web store that sells planets as a joke. A visitor loads a page of nine product cards, can register an account, log in, and click *Buy*, which hands them off to Stripe's own payment page. It is roughly 360 lines of Python in one file, five HTML templates, and one stylesheet. It exists to wire up the parts a real store needs (accounts, password hashing, a database, a payment provider) without a framework that hides how any of it works. It succeeds at that, and it also contains a login system that is broken in a way worth understanding precisely, because understanding exactly *how* it is broken teaches more than the working parts do.

Contents

1	Before anything else: the vocabulary	3
2	Orientation: what the project physically is	5
2.1	The files	5
2.2	The dependencies, and what each one is actually for	6
3	Architecture	7
3.1	The whole system in one picture	7
3.2	The seven routes	8
4	The catalog: how a product becomes a card	9
4.1	Two dictionaries and the function that joins them	9
4.2	<code>format_price()</code> , precisely	9
4.3	The template loop	10
5	The data model	11
5.1	One table, four columns	11
5.2	Where the database file lives, and why that changed	11
6	Authentication, and the defect at the centre of it	13
6.1	Registration: <code>/add_user</code>	13
6.2	Login: <code>/login</code>	13
6.3	The central defect: login is a global flag, not a session	14
7	Payments	17
7.1	The checkout route	17
7.2	Security posture of the payment flow	17
8	Configuration and secrets	19

9	The front end, briefly	19
10	Running it	20
10.1	What I verified myself	20
10.2	What I could not verify	20
11	Honest overall assessment	21

1 Before anything else: the vocabulary

This section defines every term the rest of the document leans on. Skip it if these are already familiar.

Jargon: Web application, server, client

When you open a web page, your browser (the **client**) sends a message across the network asking for a page. A program on another machine (the **server**) receives that message, decides what to say back, and sends a reply. A **web application** is that server-side program. This project is one.

Jargon: HTTP, request, response, GET, POST

HTTP is the language browsers and servers speak. The browser sends a **request**; the server sends back a **response**. Each request carries a **method** announcing intent. **GET** means "give me this page" and is what a browser does when you type an address. **POST** means "here is some data, do something with it" and is what a browser does when you submit a form. The distinction matters here: logging in is a **POST** because it submits a password.

Jargon: Status code, redirect

Every response carries a number. 200 means success. 302 and 303 mean **redirect**: "do not display anything, go ask for this other address instead." The browser follows it automatically. 500 means the server code crashed. This project uses redirects after login, after registration, and to hand the visitor to Stripe.

Jargon: Web framework, Flask

A **web framework** is a library that handles the tedious parts of being a server (parsing HTTP, matching addresses to code, building responses) so you write only your own logic. **Flask** is a deliberately small Python framework. It gives you routing and templates and little else, which is exactly why it was chosen here: nothing is hidden.

Jargon: Route, endpoint, view function

A **route** is a rule connecting a URL path to a Python function. In Flask you write `@app.route('/login')` above a function, and Flask calls that function whenever a request arrives for `/login`. The function is called a **view function**. This app has seven routes.

Jargon: Template, Jinja

A **template** is an HTML file with holes in it. **Jinja** is the engine that fills the holes with real values. `{{ planet.name }}` is a hole; Jinja replaces it with `Mercury`. `{% for ... %}` repeats a block of HTML once per item, which is how nine product cards come from one card written once.

Jargon: Database, SQL, SQLite, table, row, column

A **database** stores data so it survives the program shutting down. **SQL** is the language for querying it. **SQLite** is a database that is just a single file on disk, with no separate server to install, which is why small projects use it. Data lives in **tables**: a table is a grid, each **row** is one record (one user), each **column** is one field (the email).

Jargon: ORM, SQLAlchemy

An **ORM** (Object Relational Mapper) lets you treat database rows as ordinary Python objects instead of writing SQL by hand. **SQLAlchemy** is the standard Python ORM. Writing `User.query.filter_by(email=email).first()` produces a **SELECT** statement behind the scenes and hands back a Python object.

Jargon: Hashing, bcrypt, salt

Storing passwords as plain text is indefensible: anyone who reads the database learns every password. A **hash** is a one-way scramble. You can compute the hash of a password, but you cannot reverse the hash back into the password. To check a login you hash what was typed and compare hashes. **bcrypt** is a hashing function designed to be deliberately *slow*, so an attacker guessing billions of passwords is throttled. A **salt** is random data mixed in, so two users with the same password still get different hashes. `bcrypt` generates and embeds the salt for you.

Jargon: Session, cookie

HTTP has no memory: each request arrives knowing nothing about the last. A **cookie** is a small piece of data the server hands the browser, which the browser sends back on every later request. That is how a site remembers who you are. A **session** is the server-side notion of "this particular visitor, across their requests," normally carried by a cookie. **Remember this definition.** The central defect in this project is that it tracks login *without* using the session at all.

Jargon: Environment variable

A value handed to a program by the operating system when it starts, rather than written into the source code. Secrets belong here, because source code gets committed to version control and shared, while environment variables do not.

Jargon: Stripe, Stripe Checkout

Stripe is a payment company. **Stripe Checkout** is its hosted payment page: instead of building a card form (and inheriting the legal and security burden of touching card numbers), the app asks Stripe to create a **Checkout Session**, then redirects the buyer to a page on Stripe's own domain. The card number never touches this application. That is the single best security decision in the project.

2 Orientation: what the project physically is

2.1 The files

Every tracked file, with its role:

```
flask-online-shop
├── main.py    the entire application: config, model, catalog, all 7 routes
├── requirements.txt    5 pinned dependencies
├── .env.example    documents the 3 environment variables
├── .gitignore    excludes .env, instance/, *.sqlite3
├── README.md
├── LICENSE    PolyForm Strict: viewable, not reusable
├── templates/    Jinja HTML
│   ├── base.html    shared shell: nav, flash messages, footer
│   ├── index.html    the catalog, one card looped 9 times
│   ├── login.html
│   ├── create_user.html
│   ├── success.html
│   └── cancel.html
├── static/
│   ├── css/styles.css    321 lines, CSS variables + card components
│   └── images/    mercury.jpeg ... pluto.jpeg, 9 files
└── docs/
    ├── screenshots/app.png
    └── explainers/    this document
```

Figure 1: The complete repository. There is no test suite, no CI configuration, and no migration folder; every one of those absences is discussed later.

Everything is in one file, and that is a real decision

`main.py` holds the configuration, the database model, the product catalog, and all seven routes. For a project this size that is defensible and even helpful: a reader sees the whole system in one scroll. It is the first thing that would have to change if the project grew, and the natural split (config, models, routes, catalog data) is obvious.

2.2 The dependencies, and what each one is actually for

Package	Pin	Why it is here
Flask	3.0.3	The web framework itself: routing, templates, requests, redirects.
Flask-SQLAlchemy	3.1.1	Glue that wires SQLAlchemy into Flask's app life-cycle.
SQLAlchemy	2.0.30	The ORM. Turns the <code>User</code> class into a SQLite table.
stripe	9.12.0	Official Stripe SDK. Used for exactly one call, to create a Checkout Session.
bcrypt	4.1.3	Password hashing and verification.

Table 1: All five dependencies are pinned to exact versions, which is good practice: an unpinned install can silently pull a breaking release months later.

The dependency list is missing something the README assumes

The README instructs the reader to `cp .env.example .env` and edit it. Nothing in this project ever reads a `.env` file. There is no `python-dotenv` in `requirements.txt` and no `load_dotenv()` call anywhere in the source; I searched the whole tree for both and found neither. Flask does not read `.env` on its own (that is a feature of the `flask` CLI with `python-dotenv` installed, and this app is launched as `python main.py`). So a reader who follows the setup instructions exactly will create a `.env` file that is silently ignored, get the insecure fallback signing key and an empty Stripe key, and only discover it when checkout fails. The variables must instead be `exported` into the shell. This is a genuine README-versus-code disagreement, and the code is the truth.

3 Architecture

3.1 The whole system in one picture

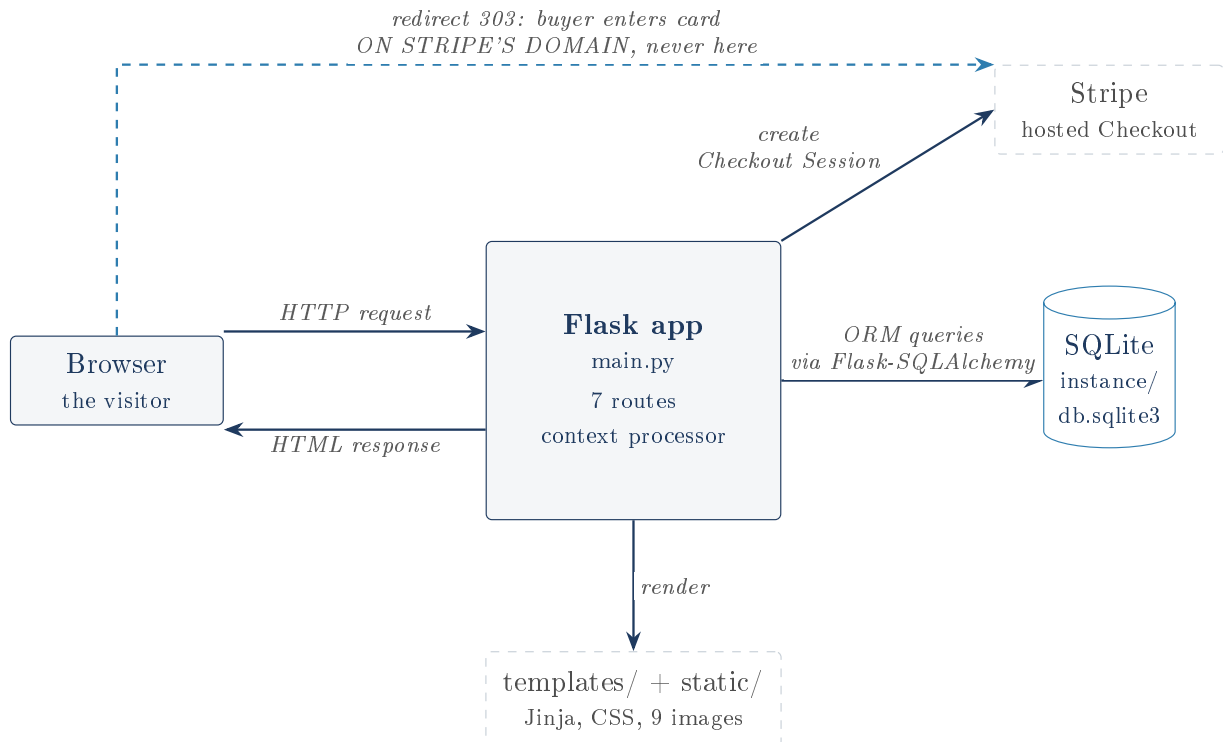


Figure 2: The system. Solid arrows are this application's own traffic; the dashed arrow is the buyer's browser leaving for Stripe. Card details flow only along the dashed path, so they never enter this codebase, this server, or this database.

Why the dashed arrow is the most important line in the diagram

Handling raw card numbers puts an application under the full weight of PCI DSS (the card industry's security standard) and makes any breach catastrophic. By redirecting to a page Stripe hosts, this project never sees a card number. A beginner project that built its own card form would be far more impressive-looking and far more dangerous. This is the right call, and it is worth being able to say why in an interview.

3.2 The seven routes

Path	Methods	Function	What it does
/	GET	home	Renders the catalog from <code>get_planets()</code> .
/create-checkout-session	POST	create_checkout_session	Builds a Stripe session for one planet, redirects to Stripe.
/success	GET	success	Static "payment worked" page.
/cancel	GET	cancel	Static "cancelled" page.
/login	GET, POST	login	Shows the form; checks the password.
/logout	GET	logout	Clears a logged-in flag.
/add_user	GET, POST	add_user	Shows the form; creates an account.

Table 2: Note the inconsistent naming: `/create-checkout-session` uses hyphens while `/add_user` uses an underscore. Cosmetic, but a reviewer will notice.

4 The catalog: how a product becomes a card

4.1 Two dictionaries and the function that joins them

Product data lives in two Python dictionaries at the top of `main.py`:

- `planet_price_dict` maps a slug (`'mercury'`) to a structure shaped exactly like a **Stripe line item**: `currency`, product name, and `unit_amount`, which Stripe requires in *cents*.
 - `PLANET_INFO` maps the same slug to display copy: `name`, `image filename`, and a `description`.
- `get_planets()` walks `PLANET_ORDER` and joins the two into the list the template loops over.

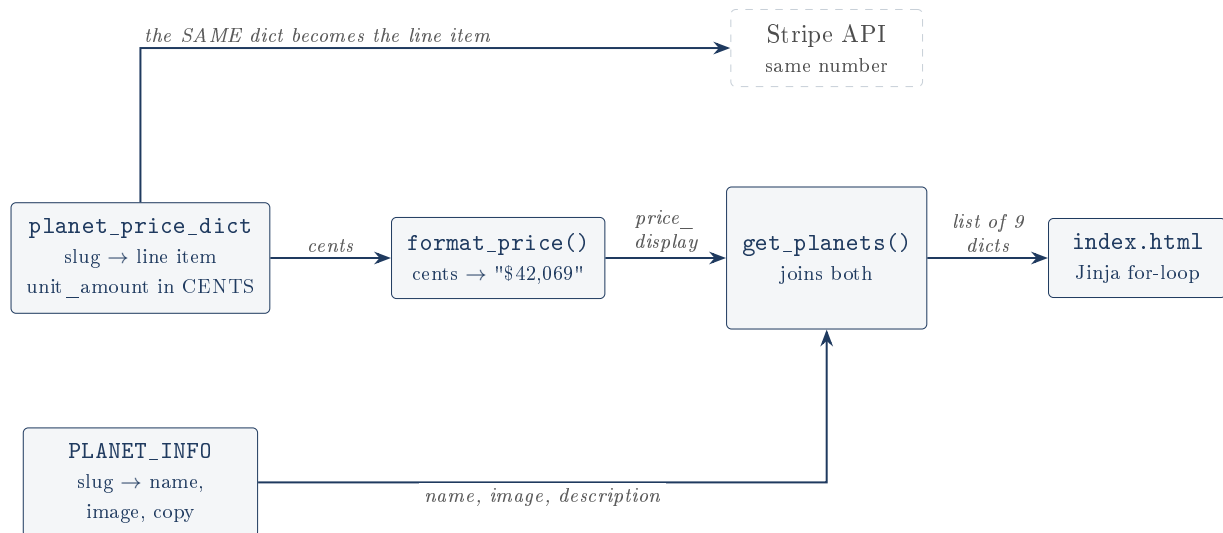


Figure 3: One number, two destinations. The price shown on the card and the price charged by Stripe are derived from the same `unit_amount`, so they cannot drift apart.

Why this indirection is not over-engineering

The README records that the template once hardcoded "Uranus: \$69,420" while the dict actually sent to Stripe said 4206900 cents, which is \$42,069. A customer would have seen one number and been charged another. That is a bug class, not a typo: the same fact was typed in two places. Deriving the display from the charge makes the bug unrepresentable. I verified the fix holds: the running app renders \$42,069 for Uranus, matching the Stripe amount exactly.

4.2 `format_price()`, precisely

```

1 def format_price(unit_amount_cents):
2     dollars = unit_amount_cents / 100
3     if dollars == int(dollars):
4         return f'${int(dollars):,}'
5     return f'${dollars:,.2f}'

```

Listing 1: The whole function. Cents in, display string out.

The `,` in the f-string inserts thousands separators. The branch drops trailing cents on whole-dollar amounts, so Mercury reads \$10,000 rather than \$10,000.00, while Earth keeps its cents at \$777,777.77. I ran `get_planets()` directly and confirmed all nine outputs; they match the underlying cent amounts in every case.

Pluto almost certainly cannot be bought

Pluto's `unit_amount` is 9999999999999999 cents, which the page renders as \$9,999,999,999,999.99. Stripe's documented maximum `unit_amount` for USD is 99999999 cents (\$999,999.99). Pluto's amount exceeds that ceiling by roughly ten million times, so the Stripe API would be expected to reject the Checkout Session with an invalid-amount error, meaning the button on "the people's favorite" fails while the other eight work.

This is labelled inferred, not verified. Confirming it requires a real Stripe test key, which this environment does not have, so I could not observe the actual API response. The claim rests on Stripe's published limit and on Pluto's amount, which I did read directly from the source. It is a joke value that was never exercised, and it is exactly the kind of thing an interviewer might click first. Earth (\$777,777.77) sits safely under the ceiling; only Pluto exceeds it.

4.3 The template loop

```

1  {% for planet in planets %}
2    <article class="product-card">
3      
5      <div class="product-body">
6        <h3 class="product-name">{{ planet.name }}</h3>
7        <div class="product-price">{{ planet.price_display }}</div>
8        <p class="product-description">{{ planet.description }}</p>
9        <form action="{{ url_for('create_checkout_session') }}" method="POST">
10         <button name="name" value="{{ planet.slug }}" type="submit" class="btn btn-buy
11         ">Buy {{ planet.name }}</button>
12       </form>
13     </div>
14   </article>
15 {% endfor %}

```

Listing 2: `index.html`: nine cards from one block of markup.

The README records that this replaced nine copy-pasted card blocks. The mechanism of the Buy button is worth noting: the `<button>` carries `name="name"` and `value="{{ planet.slug }}"`, so submitting the form POSTs a single field called `name` whose value is the slug. That is what the checkout route reads.

5 The data model

5.1 One table, four columns

```

1 class User(db.Model):
2     id = db.Column(db.Integer, primary_key=True)
3     email = db.Column(db.String(100))
4     password = db.Column(db.String(100))
5     logged_in = db.Column(db.Integer)

```

Listing 3: The entire persistence layer.

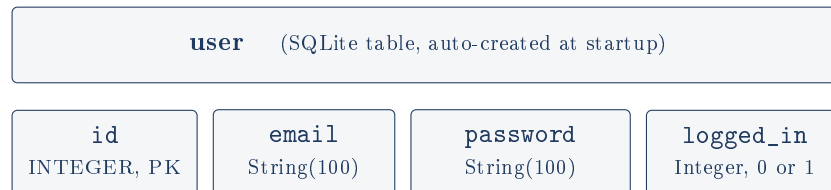


Figure 4: The complete schema. There are no other tables: no products (they are hardcoded in Python), no orders, no payments. Nothing about a purchase is ever recorded.

Jargon: Primary key

A column whose value uniquely identifies a row. `primary_key=True` on `id` makes SQLite assign each new user an automatically increasing number.

The schema has four separate defects

1. **password is declared `String(100)` but stores bytes.** `bcrypt.hashpw()` returns bytes, and that is what gets assigned. I queried the live database with `typeof(password)` and SQLite reports `blob`, not `text`. SQLite tolerates this because it is dynamically typed; a stricter database such as PostgreSQL would reject or mangle it. The consequence is visible in the login route, which carries an `isinstance(user.password, str)` guard to handle a value that may come back as either type. That guard is a symptom of the wrong column type, not a fix for it. The column should be `LargeBinary`, or the hash should be decoded to a string before storage.
2. **email has no uniqueness constraint.** Nothing stops two rows sharing an address. I registered `dup@x.com` twice against the running app and both inserts succeeded, giving two rows with the same email. Since login does `filter_by(email=...).first()`, the second account is permanently unreachable: its owner can never log in, because `first()` always returns row one. The fix is one argument: `unique=True`.
3. **logged_in is the wrong place for that fact entirely.** It is discussed at length in the next section; it is the project's central defect.
4. **No length or format validation.** `String(100)` is not enforced by SQLite, and nothing checks the email is an email.

5.2 Where the database file lives, and why that changed

```

1 base_path = os.path.dirname(os.path.abspath(__file__))
2 instance_dir = os.path.join(base_path, 'instance')
3 os.makedirs(instance_dir, exist_ok=True)
4 path_to_sql_file = os.path.join(instance_dir, 'db.sqlite3')

```

The README records that this path was once built from `sys.argv[0]`, which resolves to whatever launched the process and therefore moved depending on how the app was started. Deriving it from `__file__` (the location of the source file itself) is stable regardless of the working directory. `db.create_all()` then runs inside an **application context** at import time and creates the table if it is absent; it is idempotent, so calling it on every startup is safe.

Jargon: Application context

Flask keeps certain globals (the current app, the database handle) in a context that must be explicitly active outside a request. `with app.app_context():` is what makes `db.create_all()` legal at module level.

`create_all()` is not a migration system

It creates tables that do not exist. It will never alter one that does. Add `unique=True` to `email` tomorrow and existing databases keep the old schema silently, with no error and no upgrade. Real projects use a migration tool such as Alembic. For a demo this is an acceptable trade, but it is a limit worth naming out loud rather than being caught by.

6 Authentication, and the defect at the centre of it

6.1 Registration: /add_user

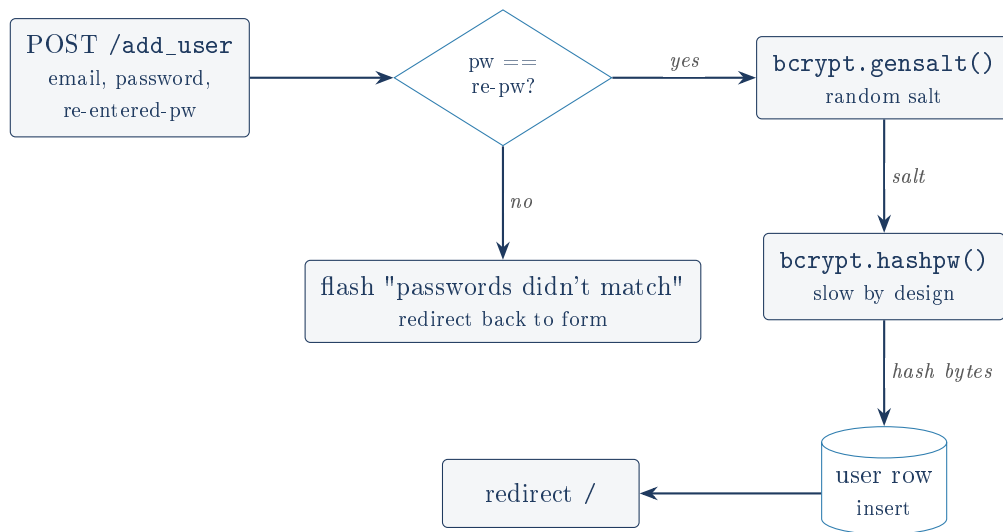


Figure 5: The registration flow. Note what is absent: no check that the email is already taken, no minimum password length, no email format validation, and no login afterwards.

The password is never stored, and that is the point

Only `bcrypt.hashpw(pw_to_bytes, bcrypt.gensalt())` reaches the database. If someone steals `db.sqlite3`, they get hashes. Because `bcrypt` is intentionally slow and each hash carries its own random salt, cracking them is expensive and cannot be done in bulk with a precomputed table. This part of the project is done correctly.

Registration validation is thinner than it looks

The registration form declares `maxlength="10"` on both password fields. That is an HTML attribute, enforced only by the browser. I posted a 60-character password directly to `/add_user`, bypassing the form: it was accepted, stored, and I then logged in with it successfully. The README lists this honestly under "What I'd do differently," and the observation is correct. Two related crashes I confirmed against the running app:

- Posting `/add_user` with the password field **missing entirely** returns HTTP 500. `request.form.get('password')` returns `None` and `bytes(None, 'UTF-8')` raises `TypeError`. A form always sends the field, so a browser user never sees this, but any direct client triggers it.
- There is no server-side email validation whatsoever; `type="email"` is, again, browser-only.

A rule worth internalising: **every client-side validation is a suggestion**. The server is the only thing that can enforce anything.

6.2 Login: /login

```

1 user = User.query.filter_by(email=email).first()
2 if user:
3     encoded_pw = bytes(pw, 'UTF-8')
4     check = bcrypt.checkpw(encoded_pw, user.password.encode('UTF-8')) if isinstance(user.
        password, str) else user.password

```

```

5     if check:
6         flash('You are now Logged In')
7         user.logged_in = 1
8         db.session.commit()
9         return redirect(url_for('home'))
10    else:
11        error = 'That password is incorrect'
12        return render_template('login.html', error=error)
13    else:
14        error = 'That email is not in our system'
15        return render_template('login.html', error=error)

```

Listing 4: The heart of the login route.

The README describes a previous version that looked the submitted email up, then always compared against user id 1's hash regardless of who was logging in. With one user that works by coincidence; with two it lets the wrong password through or rejects the right one. The current code looks up the row *by the submitted email* and checks against *that row's own hash*, which is correct.

I verified this behaves as claimed by driving the running server with two real accounts:

Attempt	Expected	Observed
Alice logs in with Alice's password	accepted	accepted, redirect to /
Bob logs in with Bob's password	accepted	accepted, redirect to /
Bob logs in with Alice's password	rejected	"That password is incorrect"
Unknown email	rejected	"That email is not in our system"

Table 3: The per-user hash check is genuinely fixed. Verified against a live server.

Two smaller issues in this route

Username enumeration. The route distinguishes "that email is not in our system" from "that password is incorrect." That tells an attacker which addresses have accounts, which is useful for targeting. Production systems return one generic message for both. For a joke storefront the stakes are nil, but the reasoning is worth knowing.

The isinstance guard. The line is long and does two jobs at once. It exists purely because the column type is wrong (see the schema section). Fixing the column would delete the guard.

No rate limiting. Nothing throttles login attempts. bcrypt's slowness is the only brake on guessing.

6.3 The central defect: login is a global flag, not a session

This is the most important thing in the codebase. Two functions carry it:

```

1  def is_logged_in():
2      return bool(db.session.query(exists().where(User.logged_in == 1)).scalar())
3
4  @app.context_processor
5  def inject_nav_state():
6      return {'logged_in': is_logged_in()}

```

Listing 5: Six lines that decide who is logged in.

Read `is_logged_in()` carefully. It asks the database a single question: **does there exist any row, anywhere, whose logged_in column equals 1?** It does not receive, and cannot receive, any information about *who is asking*. There is no cookie read, no session lookup, no user identity involved at all.

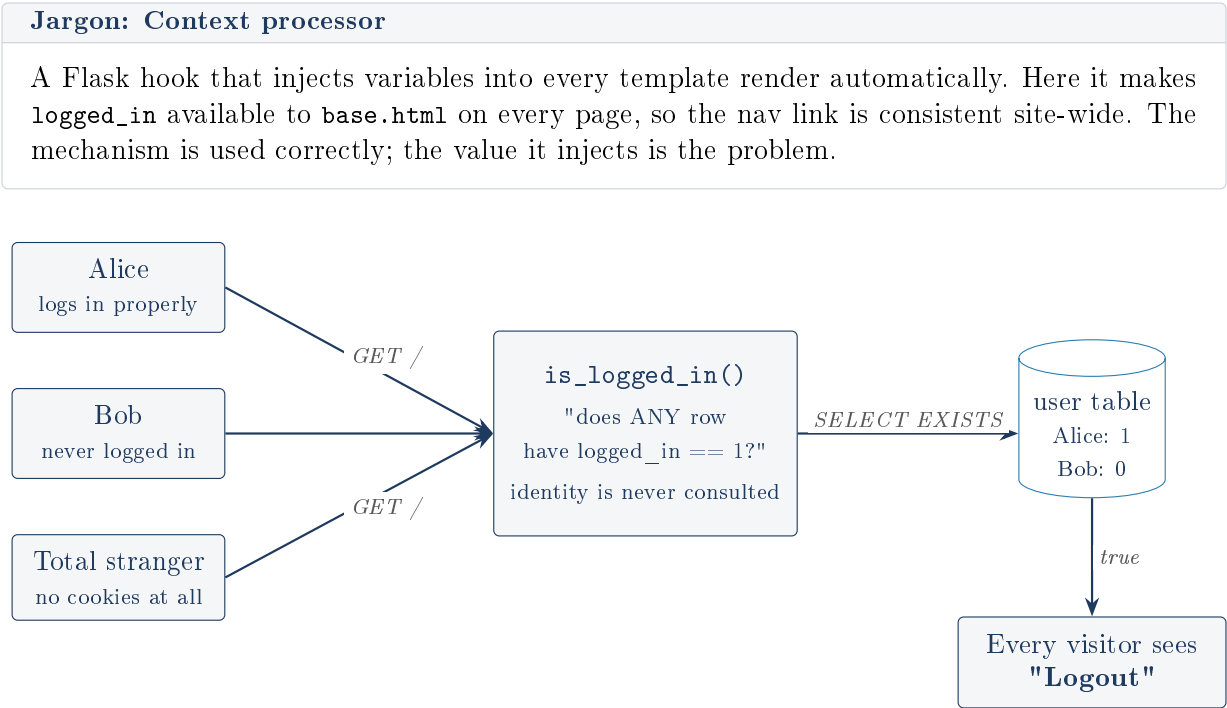


Figure 6: The defect, drawn. Three different visitors, one shared answer. Because the question never mentions who is asking, the answer cannot differ between them.

Verified experimentally: one user logging in logs everyone in

I ran the application and drove it with three independent HTTP clients, each with its own cookie jar. Results:

Visitor	Before Alice logs in	After Alice logs in
Alice’s client	shows "Login"	shows "Logout"
Bob’s client (never logged in)	shows "Login"	shows "Logout"
Brand-new client, zero cookies	shows "Login"	shows "Logout"

A visitor who has never authenticated, holding no cookie, is presented as logged in the moment any other person in the world logs in. The README describes this as "no real session isolation between browser tabs or devices," which **understates it**. It is not a tab or device issue: it is that login state is a property of the *application*, not of a visitor. Tabs and devices are the symptom; the shared global flag is the disease.

Two further consequences I confirmed:

- **/logout logs out a stranger.** The route calls `User.query.filter_by(logged_in=1).first()` and clears that row, so it logs out whichever matching row SQLite happens to return first, which is not necessarily the person who clicked. With Alice and Bob both flagged, one anonymous request to `/logout` cleared *Alice’s* row. `/logout` also requires no authentication of any kind.
- **The flag never resets.** It lives in the database, so it survives restarts. If the process dies while a user is flagged, the app comes back up believing someone is logged in, forever, until someone hits `/logout`.

Why this happened, and the honest silver lining

Flask's `session` object is imported at the top of `main.py` and is used exactly once in the whole file, at `session.pop('_flashes', None)`, to clear stale flash messages. The correct fix is to put the user's id in that session (`session['user_id'] = user.id`), because the session is backed by a signed cookie and is therefore per-visitor by construction. The `logged_in` column would then be deleted outright. The `flask-login` extension does this and more, which is what the README proposes.

The silver lining is real: the surrounding architecture is right. The context processor is the correct mechanism for site-wide nav state, and it takes its value from a single function. Change those six lines to read the session and the defect disappears without touching a template. The bug is severe but it is also *localised*, which is a consequence of the code being reasonably structured.

7 Payments

7.1 The checkout route

```

1 @app.route('/create-checkout-session', methods=['POST'])
2 def create_checkout_session():
3     planet = request.form.get('name')
4     checkout_session = stripe.checkout.Session.create(
5         line_items=[planet_price_dict[planet]],
6         mode='payment',
7         success_url=f'{YOUR_DOMAIN}/success',
8         cancel_url=f'{YOUR_DOMAIN}/cancel',
9     )
10    return redirect(checkout_session.url, code=303)

```

Listing 6: The entire payment integration.

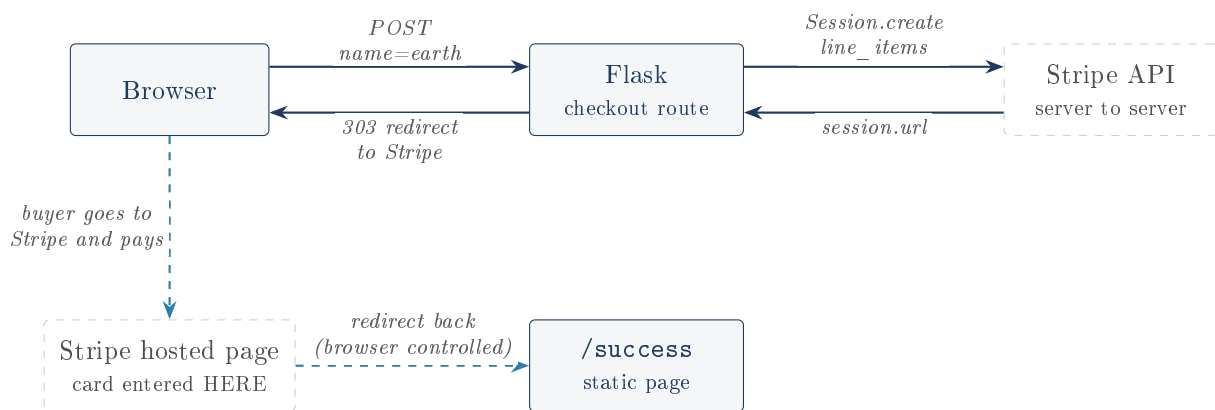


Figure 7: The checkout handoff. The application talks to Stripe’s API server-to-server to create the session, then sends the browser to Stripe’s own page. Note the return path: `/success` is reached by a plain browser redirect.

Jargon: HTTP 303

`redirect(..., code=303)` is deliberate. 303 tells the browser "your POST is done, now GET this other page." Without it a browser refresh could resubmit the POST and start a second checkout. Using 303 after a form submission is a small, correct detail.

7.2 Security posture of the payment flow

The payment flow is honest about cards and dishonest about outcomes

What is right: card data never touches this app (Stripe hosts the form); the secret key comes from the environment; the amount is server-controlled, so a buyer cannot tamper with the price by editing the form (the client sends only a slug, and the server looks the amount up).

What is wrong, verified against the running app:

1. **`/success` is a static page anyone can visit.** I requested `/success` directly, having paid nothing: HTTP 200, "payment successful." Nothing verifies a payment occurred. The route takes no session id and asks Stripe nothing.
2. **There is no webhook, so the app never learns about any payment.** A **webhook** is a callback Stripe makes to your server, server-to-server, to confirm a payment truly completed. Without one, this app’s only signal is the buyer’s browser arriving at a URL,

which the buyer fully controls. The README lists this correctly.

3. **Nothing is recorded anyway.** There is no orders table. Even a real payment leaves no trace in this system, so "who bought what" is unanswerable. This is the deeper reason the missing webhook is not a small gap: there is no place to put the answer.
4. **No login is required to buy.** The checkout route has no auth check at all. I posted to it while completely logged out and it proceeded to the Stripe call. The entire account system is therefore decorative with respect to purchasing.
5. **An unknown planet name crashes the route.** `planet_price_dict[planet]` is an unguarded dict lookup. I posted `name=krypton` and got HTTP 500 from the resulting `KeyError`. A missing `name` field does the same. It should return 400.

The takeaway an interviewer would want

"Redirect to Stripe, then trust the redirect back" is the classic beginner payment mistake, and this project makes it. The fix is a webhook plus an orders table: Stripe tells your server directly that session `cs_x` is paid, you record it, and `/success` displays only what your own database confirms. Being able to explain that in one breath is worth more than having built it.

8 Configuration and secrets

```
1 SECRET_KEY = os.environ.get('FLASK_SECRET_KEY', 'dev-only-insecure-secret-key')
2 STRIPE_SECRET_KEY = os.environ.get('STRIPE_SECRET_KEY', '')
3 YOUR_DOMAIN = os.environ.get('YOUR_DOMAIN', 'http://127.0.0.1:5000')
```

The README records that a real Stripe test key and a Flask signing key were once hard-coded in the source. Both now come from the environment, `.env` is gitignored, and `.env.example` documents each variable without containing a real value. That is the correct pattern.

Jargon: Why the signing key matters

Flask signs the session cookie with `SECRET_KEY` so a visitor cannot forge one. If the key is publicly known, anyone can mint cookies the server will trust. Here the session carries only flash messages, so the blast radius is small today; the moment login moves into the session (the correct fix above), that key becomes the thing protecting every account.

Three loose ends in configuration

1. **The insecure fallback is a footgun.** `'dev-only-insecure-secret-key'` means a deployment that forgets the variable does not crash; it runs with a publicly known signing key. The comment says "only for a quick local run," which is honest, but failing loudly in production is safer than defaulting quietly.
2. **`.env` is never loaded** (detailed earlier). The setup instructions do not work as written.
3. `app.config['SESSION_TYPE'] = 'filesystem'` **does nothing.** It is set in the `__main__` block and is a Flask-Session setting. Flask-Session is not in `requirements.txt` and is never imported, so plain Flask ignores the key entirely. It is dead configuration, and a leftover of the same confusion that produced the global login flag.

9 The front end, briefly

`base.html` is the shared shell: it loads Bootstrap 5 from a CDN, then `styles.css`, renders the nav (whose Login/Logout link is driven by the context processor), renders flashed messages, and defines the `content` block every other template fills. `styles.css` is 321 lines built on CSS variables (`--color-bg`, `--color-primary`, and so on) with reusable `product-card`, `auth-card` and `result-card` components. It is tidy and consistent.

Jargon: Flash message

Flask's mechanism for a one-shot notification that survives exactly one redirect: call `flash('...')`, and the next rendered page displays it once. It is stored in the session cookie, which is why `session.pop('_flashes', None)` at the top of `/login` exists: it clears any stale message before showing the form.

Small front-end notes

`<script src="https://js.stripe.com/v3/"></script>` is loaded on every page but never used: the redirect approach needs no client-side Stripe library. Bootstrap's CSS is pinned to 5.0.2 while its JavaScript bundle is pinned to 5.3.0-alpha3, two different versions of the same library, and neither is actually needed by anything on the page. Both are dead weight rather than bugs.

10 Running it

```
1 python3 -m venv venv
2 source venv/bin/activate
3 pip install -r requirements.txt
4
5 # The README says to copy .env and edit it. Nothing reads .env.
6 # Export the variables instead:
7 export FLASK_SECRET_KEY=$(python -c "import secrets; print(secrets.token_hex(32))")
8 export STRIPE_SECRET_KEY=sk_test_your_key_here
9
10 python main.py      # http://127.0.0.1:5000/
```

Listing 7: What actually works, as opposed to what the README says.

10.1 What I verified myself

I installed the pinned dependencies in a clean virtual environment and drove the running server. Confirmed: it boots; `GET /` returns 200 and renders exactly nine product cards; all nine displayed prices match their Stripe cent amounts; registration, login, wrong-password, unknown-email and logout all behave as the README claims; the per-user hash fix is genuine. Also confirmed, and less flattering: one user logging in makes every visitor appear logged in; duplicate emails are accepted and orphan the second account; a 60-character password bypasses the form's `maxlength`; `/success` serves 200 to someone who never paid; an unknown planet slug and a missing password field each return 500; and checkout proceeds with no login.

10.2 What I could not verify

A real Stripe checkout end to end needs a test-mode secret key that this environment does not have. With no key, `POST /create-checkout-session` reaches Stripe and fails on authentication, which confirms the route is wired correctly up to that boundary and no further. The Pluto amount claim is likewise inferred from Stripe's documented limit rather than observed.

11 Honest overall assessment

Aspect	Verdict	Note
Password hashing	Correct	bcrypt, salted, never stores plain text.
Card handling	Correct	Redirect to hosted Checkout; cards never touch this app.
Server-side pricing	Correct	Client sends a slug, server owns the amount.
Secrets in environment	Correct	Though the fallback key is a footgun and <code>.env</code> is never read.
Price display integrity	Correct	Single source; verified across all nine planets.
Template structure	Good	One shared base, one looped card.
Session handling	Broken	Login is a global flag; every visitor shares one state.
Payment confirmation	Broken	<code>/success</code> is unauthenticated and unverified; no webhook, no orders table.
Input validation	Weak	Client-side only; several inputs return 500.
Data model	Weak	No unique email, wrong column type for the hash.
Tests	Absent	No test suite at all.

What this project is, stated fairly

It is a competent beginner web application that gets the two genuinely dangerous things right (it does not store plain-text passwords and it does not touch card numbers) and then gets the state management wrong in a textbook way. The value it carries into an interview is not "I built a shop." It is: "I can show you a global mutable flag standing in for a session, explain precisely why it breaks the moment a second visitor arrives, prove it with three cookie jars, and tell you the six-line fix." That is a better story than a flawless CRUD app, provided the flaws are stated first and by the author. Every defect in this document is fixable in an afternoon, and the fact that they are localised is itself evidence the structure underneath is sound.